

ME EN 5960/6960-002: Aerospace CFD

Assignment #2

Due date: Wednesday, October 7th by 11:59pm

Submission

Assignments can only be submitted on Gradescope, which can be accessed through Canvas. Submissions will be automatically locked at the due date given above.

Use of Generative AI

This assignment expects you to utilize generative AI to progress through it. The goal of this AI use is to help you spend more time on important topics while minimizing time spent on less important/menial tasks throughout the assignment. The decision on what is a less important/menial task lies with the instructor and will be clarified in this section and throughout the assignment. You are responsible for verifying that any AI generated code implements the equations and logic you intended. Remember, we are using AI to help you learn. Please do not use it to skip the learning process. Note that sometimes, you will be learning how to best use AI as a tool for your engineering careers.

As a baseline, you may never:

1. Directly upload this assignment anywhere, including as part of an AI prompt.
2. Copy any part of this assignment and include it as part of an AI prompt. Unless explicitly banned as part of the problem, you may write a prompt in your own words based on part of the assignment.

You may always:

1. Use AI to help you understand what a problem is asking or explain some concept that is required for the problem if you follow all other AI rules. For example, if in the assignment you were asked whether a scheme is explicit/implicit, you may prompt AI with "Please explain the difference between an explicit vs. implicit scheme". You may not prompt it with the scheme you are using and ask whether it is explicit or implicit.
2. Use AI to help you understand how a programming language works, but not how to specifically solve your problem in that language. "How do I write a "for" loop in Python" is fine, "Write a CFD solver for me in Python" is not.

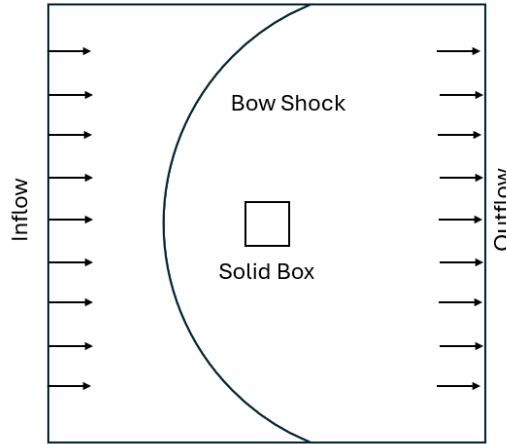
Recommended AI

You may use any AI at your disposal, but I recommend using the University of Utah ChatGPT workspace available to all students for free through <https://ai.utah.edu/tools/chatgpt/get-started.php>. This protects your data and provides a significant amount of free monthly usage. Always monitor your usage limits through settings -> usage, I cannot help you if you run out. At the time of writing, I recommend using ChatGPT 5.6 Sol Medium as it is highly effective but relatively low on resource use. Adjust appropriately as the difficulty of the problem scales.

Programming Language

I *highly* recommend using Python throughout this course, but MATLAB is acceptable if needed.

Assignment: In this assignment, you will solve the two-dimensional Euler equations to perform a simulation of $M > 1$ flow past a solid object while identifying and validating the shockwave behavior. A rough schematic of your final flow is given below.



Problem #1: Consider the full, conservative form of the Euler equations given below. These equations differ slightly from those given in Lecture 3, Slide 20 in that the energy equation solves the total energy $E = e + v^2/2$, rather than just the internal energy e .

$$\frac{\partial \rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} + \frac{\partial(\rho v)}{\partial y} + \frac{\partial(\rho w)}{\partial z} = 0$$

$$\frac{\partial(\rho u)}{\partial t} + \frac{\partial(\rho uu)}{\partial x} + \frac{\partial(\rho uv)}{\partial y} + \frac{\partial(\rho uw)}{\partial z} = -\frac{\partial p}{\partial x} + \rho f_x$$

$$\frac{\partial(\rho v)}{\partial t} + \frac{\partial(\rho vu)}{\partial x} + \frac{\partial(\rho vv)}{\partial y} + \frac{\partial(\rho vw)}{\partial z} = -\frac{\partial p}{\partial y} + \rho f_y$$

$$\frac{\partial(\rho w)}{\partial t} + \frac{\partial(\rho wu)}{\partial x} + \frac{\partial(\rho wv)}{\partial y} + \frac{\partial(\rho ww)}{\partial z} = -\frac{\partial p}{\partial z} + \rho f_z$$

$$\frac{\partial(\rho E)}{\partial t} + \frac{\partial(\rho Eu)}{\partial x} + \frac{\partial(\rho Ev)}{\partial y} + \frac{\partial(\rho Ew)}{\partial z} = \rho \dot{q} - \left(\frac{\partial pu}{\partial x} + \frac{\partial pv}{\partial y} + \frac{\partial pw}{\partial z} \right)$$

- If we want to simplify this to a spatially two-dimensional problem (in x, y), what mathematical expressions can be used to simplify our equations? In particular, what can we say about the values of w and $\partial \phi / \partial z$ where ϕ is any arbitrary variable?
- Simplify and write out the resulting equations that would be applicable to a two-dimensional problem (in x, y) assuming no body forces or heat sources. How many PDEs remain to solve?
- We will solve these equations using a numerical scheme called “Lax-Friedrichs”. This scheme was selected for the current problem because it is relatively simply to implement and is therefore instructive but note that it has shortcomings that lead to it being uncommon in real CFD codes.

Specifically, it is only first order accurate in both time and space and adds significant dissipation meaning large discontinuities (e.g., shockwaves) will be smeared out. However, it will make the coding portion much less confusing. Lax-Friedrichs is essentially just Explicit Euler with central differencing, but it replaces $\phi_{i,j}^n$ with an average of the surrounding points, which in 2D is $\phi_{i,j}^n = \frac{1}{4}(\phi_{i+1,j}^n + \phi_{i-1,j}^n + \phi_{i,j+1}^n + \phi_{i,j-1}^n)$. Typically, Explicit Euler + central difference is unconditionally unstable, but it can be shown mathematically that the replacement of this one term with the four-term average adds enough numerical dissipation to stabilize the flow for $-1 < CFL < 1$. The full set of approximations are then given by:

$$\frac{\partial \phi}{\partial t} = \frac{\phi_{i,j}^{n+1} - \frac{1}{4}(\phi_{i+1,j}^n + \phi_{i-1,j}^n + \phi_{i,j+1}^n + \phi_{i,j-1}^n)}{\Delta t}$$

$$\frac{\partial \phi}{\partial x} = \frac{\phi_{i+1,j}^n - \phi_{i-1,j}^n}{2\Delta x}$$

$$\frac{\partial \phi}{\partial y} = \frac{\phi_{i,j+1}^n - \phi_{i,j-1}^n}{2\Delta y}$$

Write the update equations that result from applying these approximations to your simplified set of PDEs, organizing $n + 1$ terms on the left and n terms on the right.

- d) Your equations should give you a fully explicit scheme where values at the next timestep could be calculated only using data from the previous timestep. However, note that you are solving for five variables including ρ, u, v, P and E , but you only have four equations to solve. You can close this problem by including an equation of state to calculate the pressure. You can thermodynamically show that the usual equation of state ($p = \rho RT$) can be more conveniently written for this problem as $p = (\gamma - 1) \left[\rho E - \frac{1}{2} \rho (u^2 + v^2) \right]$. Express this equation in terms of i, j, n notation in a way that allows you to calculate the pressure, assuming constant γ .

Problem #2: Now that you have all the equations, write a program to solve them. Note that because this is two dimensional, you will now have to generate a two-dimensional array to store your data at each timestep and use three nested loops (time, x , y) to update your values. You should have a square domain where $0 < x < 1$ and $-0.5 < y < 0.5$. Your code should have the capability to adjust the number of points in each direction but begin with $n_x = 101$ and $n_y = 101$ which should give $\Delta x = \frac{x_f - x_i}{n_x - 1} = .01$ and the same for Δy . You may pick your own CFL number, noting that the scheme is stable for $CFL < 1$, but note that you will have to calculate an allowable Δt at every point at every time for both the x and y components and take the minimum for the next timestep (see Lecture 7, Slide 19). Recall that we are now solving a real compressible flow, so the characteristic velocities that define the CFL number have the speed of sound added to the velocity components. Assume $\gamma = 1.4$.

For initial conditions, use:

$$\rho(x, y, 0) = 1.0$$

$$u(x, y, 0) = 2000.0$$

$$v(x, y, 0) = 0.0$$

$$p(x, y, 0) = 101325.0$$

$$E(x, y, 0) \rightarrow \text{calculate from EOS}$$

Note in the above that the variables are all related, so one of them is set based on the others. In this case, the initial value of E should be calculated from the equation of state.

For the upper and lower boundary, specify everything with Dirichlet boundary conditions with the following values. For the left boundary, we will specify an inlet with the same Dirichlet boundary conditions:

$$\rho(x, -0.5, t) = \rho(x, 0.5, t) = \rho(0, y, t) = 1.0$$

$$u(x, -0.5, t) = u(x, 0.5, t) = u(0, y, t) = 2000.0$$

$$v(x, -0.5, t) = v(x, 0.5, t) = v(0, y, t) = 0.0$$

$$p(x, -0.5, t) = p(x, 0.5, t) = p(0, y, t) = 101325.0$$

$$E(x, -0.5, t) = E(x, 0.5, t) = E(0, y, t) \rightarrow \text{calculate from EOS}$$

For the right boundary, we will set a zero-gradient Neumann boundary condition as an outlet for all quantities ($\frac{\partial \phi}{\partial x} = 0$). Discretize this equation with a first order backward difference for each quantity. This allows you to solve for the rightmost points.

$$\rho_{n_x} = \rho_{n_x-1}$$

$$u_{n_x} = u_{n_x-1}$$

$$v_{n_x} = v_{n_x-1}$$

$$E_{n_x} = E_{n_x-1}$$

$$p_{n_x} = p_{n_x-1}$$

Because our boundary conditions equal our initial conditions and there is nothing inside the domain, a successful code at this point is one where the variables inside the domain do not change significantly over time. Verify that you see this expected behavior. **AI USAGE:** You **should** heavily utilize AI for this problem, but it may not write any code for you. Instead, it should act as an assistant that can help you understand how to properly structure your code and help you find bugs. If you are unclear on how/when certain quantities should be calculated, how to structure your arrays, etc., you may ask your AI assistant for guidance. Additionally, once you have a significant portion of the code written but want to check that you've gotten, for example, all the i, j indices correct, you may upload it to the AI and ask it to identify bugs. However, remember that all actual code must be written by you. Secondary features like plotting/creating videos that are not core to obtaining a solution to the equations can be written entirely by AI. Do not move on until you've had the AI assess your code and any major issues have been resolved.

Problem #3: Now that you have a functional solver, we are going to add one more component that will give us a more interesting flow. You will implement a solid, adiabatic, slip-wall box in the center of the

domain which will lead to the formation of a bow shock. The box should lie between $0.45 < x < 0.55$ and $-0.05 < y < 0.05$. You will need to 1) identify points inside the box and exclude them from your state update since they no longer represent an evolving fluid and 2) apply boundary conditions at the edges of your box. For an adiabatic box with slip-wall conditions, the value of the velocity normal to the surface should be equal to the negative of itself, while the value of all other quantities (call them f) should equal the value just outside the box, as given by the following:

Along the Left and Right Wall

$$u(i, j, t) = -u(i, j, t)$$

$$f(i, j, t) = f(i, j, t)$$

Along the Top and Bottom Wall

$$v(i, j, t) = -v(i, j, t)$$

$$f(i, j, t) = f(i, j, t)$$

AI USAGE: You can rely heavily on AI to help walk you through this portion if you are unclear how to proceed. Follow the AI usage guidelines from problem #2 to develop these features.

Once this is completed, run your simulation until it reaches a sufficiently steady state (how can you assess this?). Comment on what you see with respect to what we've discussed about compressible flows. In particular:

- a) What happens to the various properties across the shockwave, and does it match expectations?
- b) How does the thickness of the shock compare to what we expect from theory?
- c) How do property changes across the normal shock compare to those across the oblique shock sections and is that supported by theory?